

CSCD602:

Advanced Software Engineering

Session 5 – Software Design & Architecture

By

Solomon Mensah (PhD)

Dept. of Computer Science

smensah03@ug.edu.gh



UNIVERSITY OF GHANA

Course Agenda

- **01. UML Diagrams**
Use Case, Class, Sequence, Activity, Component
- **02. Architectural & Design Patterns**
MVC, Microservices, Layered Architecture
- **03. SOLID Design Principles**
Five foundational principles for clean code
- **04. User-Centred Design**
Personas, user journeys, accessibility
- **05. Usability Testing**
Methods, metrics, and best practices
- **06. Iterative Development**
Integrating feedback through Agile cycles
- **07. Case Study**
End-to-end design of a real-world system

UML Diagrams

- Unified Modelling Language — a standardised visual notation for specifying, visualising, and documenting software systems. UML bridges the gap between problem domain and technical implementation.

What is UML?

Definition: UML (Unified Modelling Language) is a general-purpose, standardised modelling language in software engineering, providing a way to visualise a system's architectural blueprints.

Purpose: UML helps teams communicate, document, and design complex systems before writing a single line of code — reducing ambiguity and costly misunderstandings.

Adoption: Standardised by the OMG (Object Management Group), UML is used across industries including finance, healthcare, and enterprise software.

Two Categories: Structural diagrams show the static composition of a system; Behavioural diagrams depict the dynamic interactions and state changes at runtime.

Structural UML Diagrams

Class Diagram

- Shows classes, attributes, methods and their relationships
- Key relationships: Association, Aggregation, Composition, Inheritance
- Example: A 'Customer' class has-many 'Order' objects
- Notations: + public, - private, # protected
- Most widely used UML diagram in OOP design

Component Diagram

- Depicts how components are wired together to form a larger system
- Shows provided and required interfaces between components
- Example: PaymentGateway component depends on BankAPI component
- Useful for defining system boundaries and dependencies
- Critical for microservice architecture documentation

Cont'd - Package Diagram

- Organises model elements into groups called packages
- Shows dependencies between packages with dashed arrows
- Example: 'UI' package depends on 'Services' package
- Helps manage names pacing and module structure
- Reduces visual complexity of large class diagrams

Behavioural UML Diagrams

Use Case Diagram

- Captures functional requirements from the user's perspective
- Actors represent external entities (users or systems)
- Use cases describe what the system does, not how
- Relationships: Include, Extend, Generalisation
- Example: 'Customer' actor → 'Place Order' use case

Behavioural UML Diagrams – Cont'd

Sequence Diagram

- Shows how objects interact over time (time runs top to bottom)
- Lifelines represent participants; arrows show messages
- Synchronous (filled arrowhead) vs. asynchronous messages
- Example: Login flow — Browser → Server → Database
- Excellent for API design and debugging call flows

Behavioural UML Diagrams – Cont'd

Activity Diagram

- Models the flow of control or data through a process
- Decision nodes for branching; fork/join for concurrency
- Swimlanes assign responsibilities to roles or systems
- Example: Order processing workflow with payment validation
- Bridges business process modelling and implementation logic

Design & Architectural Patterns

Design & Architectural Patterns

- Proven, reusable solutions to recurring software design challenges. Patterns encode expert knowledge and best practices, enabling teams to build scalable, maintainable systems more efficiently.

MVC — Model-View-Controller

Model: Manages application data, business logic and rules. Notifies the View of state changes. Does not depend on View or Controller.

View: Renders the UI and presents data from the Model. Forwards user input events to the Controller. Remains passive and decoupled from business logic.

Controller: Acts as the intermediary — receives user input, manipulates the Model, and determines which View to display next.

Benefits: Separation of concerns simplifies testing and parallel development. Each component can evolve independently, reducing coupling.

Examples: Ruby on Rails, Django, ASP.NET MVC, Spring MVC. Modern variants include MVP (Model-View-Presenter) and MVVM for richer UIs.

MVC Flow

1. . User interacts with the View
2. View notifies the Controller
3. Controller updates the Model
4. Model notifies the View of changes
5. View re-renders updated data

Key Benefit: Testability — each layer is independently unit-testable

Used By: Rails, Django, Laravel, Spring

Microservices Architecture

Definition: An architectural style that structures an application as a collection of small, independently deployable services, each running in its own process and communicating via APIs.

Decomposition: Services are organised around business capabilities (e.g., User Service, Order Service, Payment Service). Each has its own database to ensure loose coupling.

Communication: Services communicate using REST, gRPC, or message brokers (Kafka, RabbitMQ). Synchronous vs. asynchronous patterns have different trade-offs.

Deployment: Services are containerised (Docker) and orchestrated (Kubernetes). This enables independent scaling — scale only what needs it.

Trade-offs: Increased operational complexity (distributed tracing, service mesh). Requires DevOps maturity. Risk of distributed system failures (partial failures, latency).

Microservices vs Monolith

Monolith: Single deployable, shared DB, simpler to develop initially

Microservices: Independent deployment, own DB per service, fault isolation

When to choose Microservices:

- Large teams working in parallel
- Need for independent scaling
- Different tech stacks per service
- High availability requirements

Layered (N-Tier) Architecture

Concept: Organises code into horizontal layers where each layer has a distinct role. Each layer only communicates with the layer directly below it, creating a clear dependency hierarchy.

Presentation Layer: Handles all user interface logic. Displays data and captures user input. Should contain NO business logic.

Business Logic Layer: Implements application rules, calculations, and workflows. The core of the application — independent of UI and data storage concerns.

Data Access Layer: Abstracts database operations. Uses patterns like Repository or DAO. Decouples the business layer from specific database technologies.

Benefits & Pitfalls: Highly maintainable and testable. Risk of 'sinkhole' anti-pattern: requests passing through layers that add no value, hurting performance.

Common Layer Stack

- ① Presentation (UI/API controllers)
- ② Application (use cases / DTOs)
- ③ Domain (business entities / rules)
- ④ Infrastructure (DB, email, cache)

Rule: Dependencies point downward only

Testing: Mock the layer below to isolate logic

Example stacks: .NET Clean Architecture, Java Spring

SOLID Design Principles

- Five object-oriented design principles introduced by Robert C. Martin ('Uncle Bob'). SOLID principles guide developers in writing code that is easy to maintain, extend, and test over time.

SOLID Principles — S, O, L

S — Single Responsibility

- A class should have only ONE reason to change
- Each class encapsulates ONE concern or responsibility
- Bad: 'UserService' handles auth, email AND reporting
- Good: Separate AuthService, EmailService, ReportService
- Reduces coupling; makes classes easier to test and reuse
- Aligns with 'cohesion' — keeping related things together

O — Open/Closed Principle

- Software entities should be OPEN for extension, CLOSED for modification
- Add new behaviour by extending, not editing existing code
- Achieved via interfaces, abstract classes, and polymorphism
- Example: Add a new PaymentMethod subclass without changing PaymentProcessor
- Prevents regression bugs introduced when editing tested code

L — Liskov Substitution

- Subtypes must be substitutable for their base types without altering program correctness
- If S extends T, any code using T must work correctly with S
- Violated when a subclass throws exceptions or weakens behaviour the base class promises
- Example: Square extending Rectangle breaks LSP if setWidth/setHeight are independent
- Enforces correct use of inheritance

SOLID Principles — I & D

I — Interface Segregation Principle

- Clients should not be forced to depend on methods they do not use
- Prefer many small, specific interfaces over one large 'fat' interface
- Bad: One 'Printer' interface with print(), scan(), fax() — basic printers can't fax
- Good: IPrinter, IScanner, IFax — classes implement only what they need
- Reduces unnecessary coupling between unrelated parts of the system
- Improves code readability and supports the Single Responsibility Principle
- Commonly applied in plugin architectures and framework design

SOLID Principles — I & D

D — Dependency Inversion Principle

- High-level modules should not depend on low-level modules; both should depend on abstractions
- Abstractions should not depend on details; details should depend on abstractions
- Bad: OrderService directly instantiates MySQLRepository
- Good: OrderService depends on IOrderRepository interface; inject the concrete implementation
- Enables Dependency Injection (DI), a core pattern in modern frameworks (Spring, .NET Core, Angular)
- Makes code testable — inject mock implementations in unit tests
- Underpins Inversion of Control (IoC) containers

Principles of User-Centered Design

Principles of User-Centred Design

UCD is an iterative design process where the needs, wants, and limitations of end users are given extensive attention at each stage of the design process — ensuring the product is built FOR the user, not AT the user.

User-Centred Design — Core Principles

Understand Users Early: Conduct user research (interviews, surveys, observation) before designing. Build empathy maps and personas to represent real user segments, goals, and pain points.

Involve Users Throughout: Co-design sessions, participatory workshops, and regular feedback checkpoints ensure the product evolves with real user needs — not assumptions.

Iterate Rapidly: Use low-fidelity prototypes (sketches, wireframes) to test ideas cheaply before investing in full development. Fail fast, learn fast.

Evaluate Against User Needs: Validate designs with real users through usability testing. Measure task success rates, error rates, and satisfaction scores — not just aesthetic appeal.

Accessibility & Inclusivity: Design for diverse users including those with disabilities (WCAG 2.2 guidelines). Inclusive design benefits everyone — e.g., captions help in noisy environments too.

ISO 9241-210 UCD Activities

- ① Understand context of use
 - ② Specify user requirements
 - ③ Produce design solutions
 - ④ Evaluate against requirements
- ↻ Repeat until requirements met
-

Key Deliverables:

- User personas & journey maps
- Wireframes & prototypes
- Usability test reports

Personas & User Journey Mapping

User Personas

- A persona is a fictional, research-based representation of a user archetype
- Includes: Name, age, job, goals, frustrations, tech literacy, quote
- Based on patterns from real user interviews — not stereotypes
- Example: 'Sarah, 34, nurse — needs quick data entry during rounds, low tolerance for complex UIs'
- Prevents 'designing for yourself' — keeps team empathetic to real users
- Primary persona: drives design decisions; Secondary: edge cases to accommodate
- Update personas as new user research is gathered

Personas & User Journey Mapping

User Journey Maps

- Visualises a user's end-to-end experience achieving a goal across time and touchpoints
- Stages: Awareness → Discovery → Use → Support → Loyalty
- Each stage captures: Actions, Thoughts, Emotions, Pain Points, Opportunities
- Reveals friction points invisible to the development team
- Example: Map the 'return a product' journey — frustrated by 7-step form = redesign opportunity
- Empathy curve: plot emotional highs and lows across the journey
- Drives feature prioritisation based on highest user pain

Usability Testing

Usability Testing

Usability testing is an evaluation method where real users attempt to complete realistic tasks with a product. It uncovers interface problems, informs design decisions, and validates that the design meets user goals.

Usability Testing — Methods & Metrics

Moderated Testing

- session — guides, probes, and clarifies
- Think-aloud protocol: users verbalise thoughts while completing tasks
- Rich qualitative insights into WHY problems occur
- Typically 5–8 participants (Nielsen: 5 users find 85% of usability issues)
- Conducted in-person or via screen-sharing tools (Zoom, Lookback)
- More expensive but yields deeper understanding

Unmoderated Testing

- Users complete tasks independently — no facilitator present
- Tools: UserTesting.com, Maze, Optimal Workshop
- Faster and cheaper — can reach 50+ participants quickly
- Captures natural, uninfluenced behaviour
- Best for validating specific task flows at scale
- Less context on WHY — combine with moderated sessions

Key Usability Metrics

- Task Completion Rate: % of users who successfully finish a task
- Time on Task: How long to complete — benchmark against expert performance
- Error Rate: Number of mistakes made per task
- SUS (System Usability Scale): 10-item survey; score ≥ 68 = above average
- NPS (Net Promoter Score): Likelihood to recommend (0–10 scale)
- Learnability: Performance improvement after repeated use

Integrating User Feedback

Integrating User Feedback in Iterative Development

Agile and iterative development place continuous user feedback at the heart of the product lifecycle. Each sprint delivers working software that can be tested, evaluated, and refined based on real user behaviour

Integrating Feedback — Agile & Iterative Cycles

Sprint Review & Demo: At the end of each sprint, stakeholders and users see working software. Real reactions — positive and negative — feed directly into the product backlog for prioritisation.

Continuous User Testing: Don't wait for a final product. Test paper prototypes in sprint 1, interactive wireframes in sprint 3, and production builds in sprint 6. Problems found early cost 100x less to fix.

Analytics & Behavioural Data: Quantitative signals (click heatmaps, drop-off rates, session recordings via Hotjar or FullStory) reveal what users DO, complementing what they SAY in interviews.

Backlog Refinement: User feedback is converted into user stories, bugs, or improvement tasks. Stories are refined with acceptance criteria, estimated, and prioritised by business value vs. effort.

Definition of Done (DoD): Explicitly include usability criteria in DoD: 'Feature achieves $\geq 80\%$ task completion in usability test with 5 participants before release'.

Feedback Loop Cycle

- Plan → Build → Test → Review
- → Collect user feedback
- → Analyse and prioritise
- → Update backlog
- → Next sprint (repeat)
- _____
- Tools: Jira, Confluence, Hotjar, Maze, UserVoice, Intercom
- Frequency: Every 1–2 weeks (sprint cadence)

Case Study: E-Commerce Platform Redesign

Applying UML, architectural patterns, SOLID principles, and user-centred design to a real-world system. Follow the complete design journey from requirements through testing and iteration.

Case Study — ShopEase Platform Redesign

Problem: ShopEase had a 68% cart abandonment rate and a SUS score of 52 (below average). User interviews revealed confusing checkout, slow search, and poor mobile experience.

Architecture Decision: Migrated from a legacy monolith to a Microservices architecture. Separate services for: Product Catalogue, Cart, Payments, User Accounts, and Notifications.

UML Deliverables: Use Case diagrams defined 12 key user stories. Sequence diagrams documented the checkout and payment flows. Class diagrams modelled the Product, Order, and Cart domains.

UCD Process: 3 persona types identified (Casual Browser, Repeat Buyer, Gift Purchaser). Journey maps revealed 5 critical pain points in checkout. Co-design sessions produced 3 low-fi prototypes.

Outcomes: Post-launch: Cart abandonment reduced to 41%. SUS score improved to 74. Mobile conversions up 33%. SOLID principles enabled two new payment providers to be added in 3 days.

Results at a Glance

- Cart Abandonment: 68% → 41%
- SUS Score: 52 → 74 (Good)
- Mobile Conversion: +33%
- Page Load: 4.2s → 1.1s
- New Feature Delivery: 2x faster
- Patterns Used: Microservices, MVC for frontend, Repository pattern
- Principles: Full SOLID compliance, WCAG 2.1 AA accessibility

Key Takeaways

- **UML:** is a shared visual language that aligns teams before, during, and after development. Use the right diagram for the right question.
- **Patterns:** provide proven blueprints. MVC separates concerns; Microservices enable scale; Layered architecture enforces discipline.
- **SOLID:** principles reduce technical debt. Code that follows SOLID is easier to test, extend, and hand off to other developers.
- **UCD:** ensures you build the RIGHT thing. Empathy, research, and personas keep the user at the centre of every design decision.
- **Test early & iterate:** Usability testing at every sprint is cheaper than fixing a broken product post-launch. Let data drive decisions.

Thank you

